



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ

ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА

СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

**Отчет по лабораторной работе № 1
по дисциплине Методы машинного обучения в АСОИУ**

Тема работы: "Обработка признаков, часть 1"

Выполнил:

Студент группы ИУ5Ц-21М
Папин А.В.

(дата, подпись)

Проверил:

Преподаватель
Гапанюк Ю.Е.

(дата, подпись)

Москва, 2026

СОДЕРЖАНИЕ ОТЧЕТА

Цель работы	3
Задание	3
Ход работы.....	4
Разведочный анализ данных (EDA).....	4
Устранение пропусков в данных	7
Кодирование категориальных признаков	11
Нормализация числовых признаков	14
Вывод.....	18

Цель работы

Целью лабораторной работы является изучение продвинутых способов предварительной обработки данных для дальнейшего формирования моделей

Задание

1. Выбрать набор данных (датасет), содержащий категориальные и числовые признаки и пропуски в данных. Для выполнения следующих пунктов можно использовать несколько различных наборов данных (один для обработки пропусков, другой для категориальных признаков и т.д.) Просьба не использовать датасет, на котором данная задача решалась в лекции.

2. Для выбранного датасета (датасетов) на основе материалов лекций решить следующие задачи:

- 2.1. устранение пропусков в данных;
- 2.2. кодирование категориальных признаков;
- 2.3. нормализация числовых признаков.

Ход работы

Разведочный анализ данных (EDA)

Для выполнения лабораторной работы будем использовать датасет, исследующий вопрос, заменят ли электромобили (EV) бензиновые и дизельные автомобили. Датасет содержит данные о продажах автомобилей, инфраструктуре зарядки, экономических показателях и политических индикаторах за период 2010-2025 гг.

Ссылка на датасет – <https://www.kaggle.com/datasets/aryanmdev/will-evs-replace-petrol-cars>

Первичный анализ данных

```
Информация о датасете:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1200 entries, 0 to 1199
Data columns (total 22 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   country                              1200 non-null   object
1   region                              1200 non-null   object
2   year                                1200 non-null   int64
3   vehicle_segment                     1200 non-null   object
4   powertrain_type                     1200 non-null   object
5   ev_sales                            1200 non-null   int64
6   petrol_car_sales                    1200 non-null   int64
7   diesel_car_sales                    1200 non-null   int64
8   total_vehicle_sales                 1200 non-null   int64
9   ev_market_share                     1200 non-null   float64
10  charging_stations                   1200 non-null   int64
11  fast_chargers_share                 1200 non-null   float64
12  avg_ev_range_km                     1200 non-null   int64
13  fuel_price_usd_per_liter            1200 non-null   float64
14  electricity_price_usd_per_kwh       1200 non-null   float64
15  gdp_per_capita                       1200 non-null   int64
16  urban_population_percent             1200 non-null   float64
17  co2_emissions_transport_mt          1200 non-null   float64
18  ev_subsidy_usd                      1200 non-null   int64
19  emission_regulation_score           1200 non-null   float64
20  ev_growth_rate_yoy                  1200 non-null   float64
21  is_ev_dominant                      1200 non-null   int64
dtypes: float64(8), int64(10), object(4)
memory usage: 206.4+ KB
```

И выполнили описательную статистику со следующими информациями:

Датасет (1200 наблюдений, 2010–2025) показывает быстрый, но неравномерный рост рынка EV.

- Средняя доля EV (ev_market_share): 6.33%, медиана: 0.83%, максимум: 95%

- EV доминируют (is_ev_dominant) лишь в 1.8% наблюдений

– Средний рост продаж EV (ev_growth_rate_yoy): 64%, медиана: 44%, максимум: 300%

Технологический прогресс:

– Средний запас хода (avg_ev_range_km): 266 км (рост от 106 км до 507 км)

Инфраструктура:

– Медиана зарядных станций (charging_stations): 4 090, верхний квартиль: 22 518, максимум: 4.34 млн

– Доля быстрых зарядок (fast_chargers_share, медиана): 12.9%

Экономика и политика:

– Средний ВВП на душу (gdp_per_capita): \$35 652

– Средняя субсидия (ev_subsidy_usd): \$2 911, максимум: 8 952

– Средний индекс регулирования (emission_regulation_score): 58.8 (макс. 95.6)

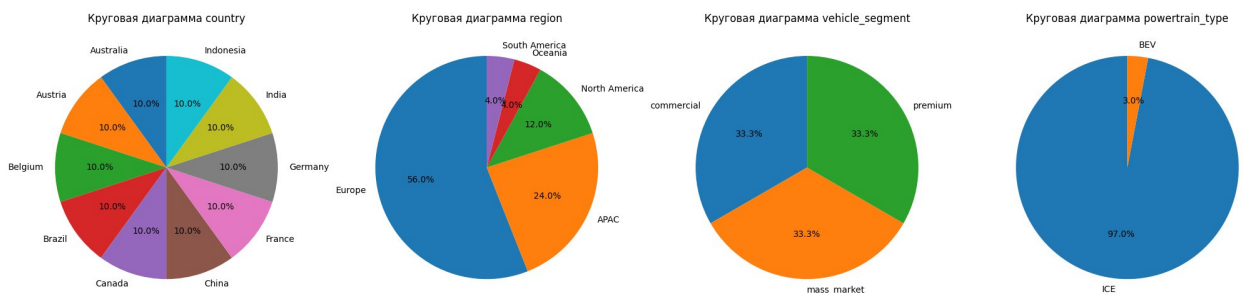
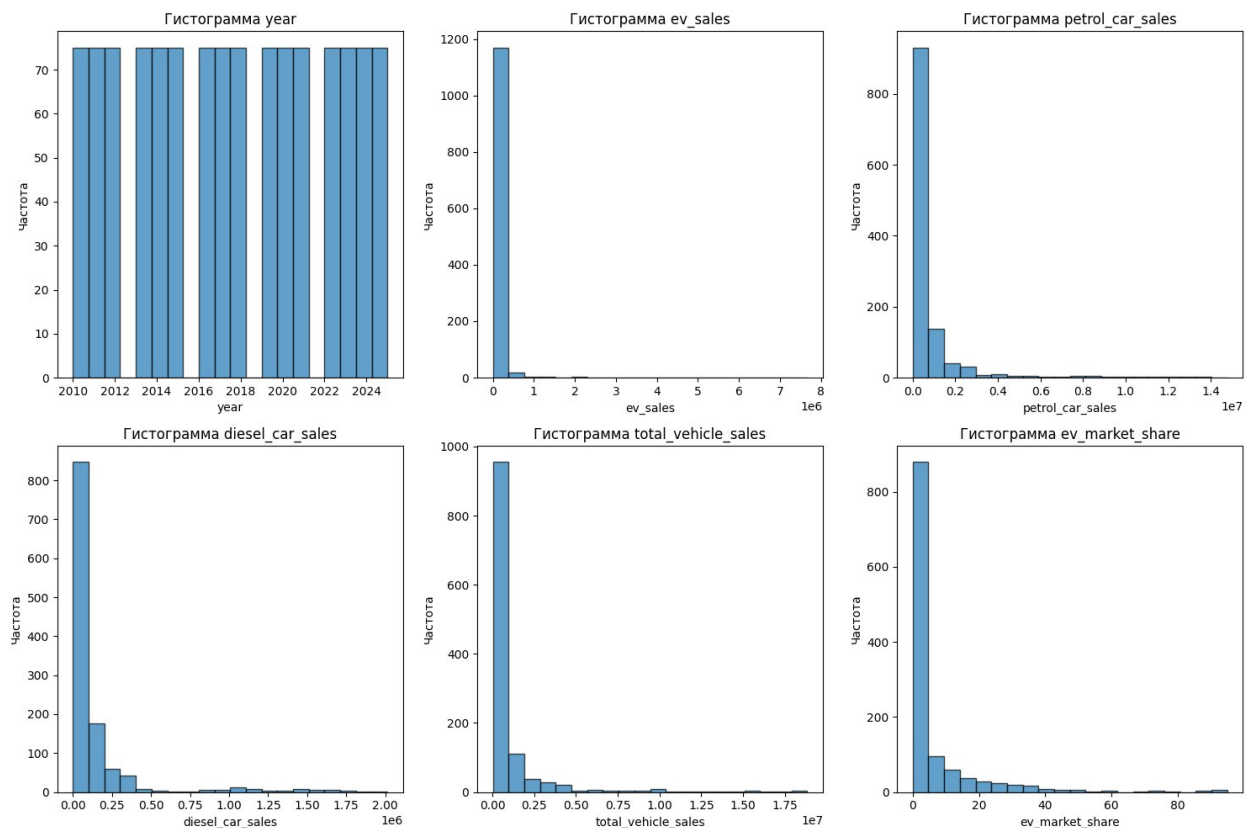
После анализа описательной статистики можно сделать промежуточный вывод по датасету – EV быстро растут ($\approx 64\%$ в год), но глобально остаются меньшинством (средняя доля 6.33%); переход концентрируется в странах с высокой инфраструктурой, доходами и жёсткой экологической политикой.

Рассмотрели и проверили пропущенные значения

```
1 print("\nПропуски в данных:")
2 print(df.isnull().sum())
```

```
Пропуски в данных:
country              0
region              0
year                0
vehicle_segment      0
powertrain_type      0
ev_sales             0
petrol_car_sales     0
diesel_car_sales     0
total_vehicle_sales  0
ev_market_share      0
charging_stations    0
fast_chargers_share  0
avg_ev_range_km      0
fuel_price_usd_per_liter  0
electricity_price_usd_per_kwh  0
gdp_per_capita       0
urban_population_percent  0
co2_emissions_transport_mt  0
ev_subsidy_usd       0
emission_regulation_score  0
ev_growth_rate_yoy   0
is_ev_dominant       0
dtype: int64
```

Пропущенных значений нет, переходим к следующему – визуализацию распределения числовых признаков (гистограммы) и Визуализация категориальных признаков (круговые диаграммы в виде сетки)



По этому графику можно заметить, что количество информации автомобилей преобладает в европейской части по сравнению с остальными регионами. Сегмент автомобиля получилось сбалансированным, а тип двигателя автомобиля преобладает ДВС (Internal Combustion Engine) по сравнению с электродвигателем (Battery Electric Vehicle). Со страной получилось сбалансированной.

Устранение пропусков в данных

Анализ наличия пропущенных значений

Поскольку у нас нет пропущенных значений, поэтому искусственно создадим пропущенные значения с 15% в произвольных выбранных колонках – 'ev_sales', 'gdp_per_capita', 'fuel_price_usd_per_liter', 'country', 'vehicle_segment'.

```
Оригинальный датасет - пропусков:
country                                0
region                                0
year                                  0
vehicle_segment                        0
powertrain_type                       0
ev_sales                              0
petrol_car_sales                      0
diesel_car_sales                      0
total_vehicle_sales                   0
ev_market_share                       0
charging_stations                     0
fast_chargers_share                   0
avg_ev_range_km                       0
fuel_price_usd_per_liter              0
electricity_price_usd_per_kwh         0
gdp_per_capita                        0
urban_population_percent               0
co2_emissions_transport_mt            0
ev_subsidy_usd                        0
emission_regulation_score              0
ev_growth_rate_yoy                    0
is_ev_dominant                        0
dtype: int64
Всего пропусков в оригинальном датасете: 0
```

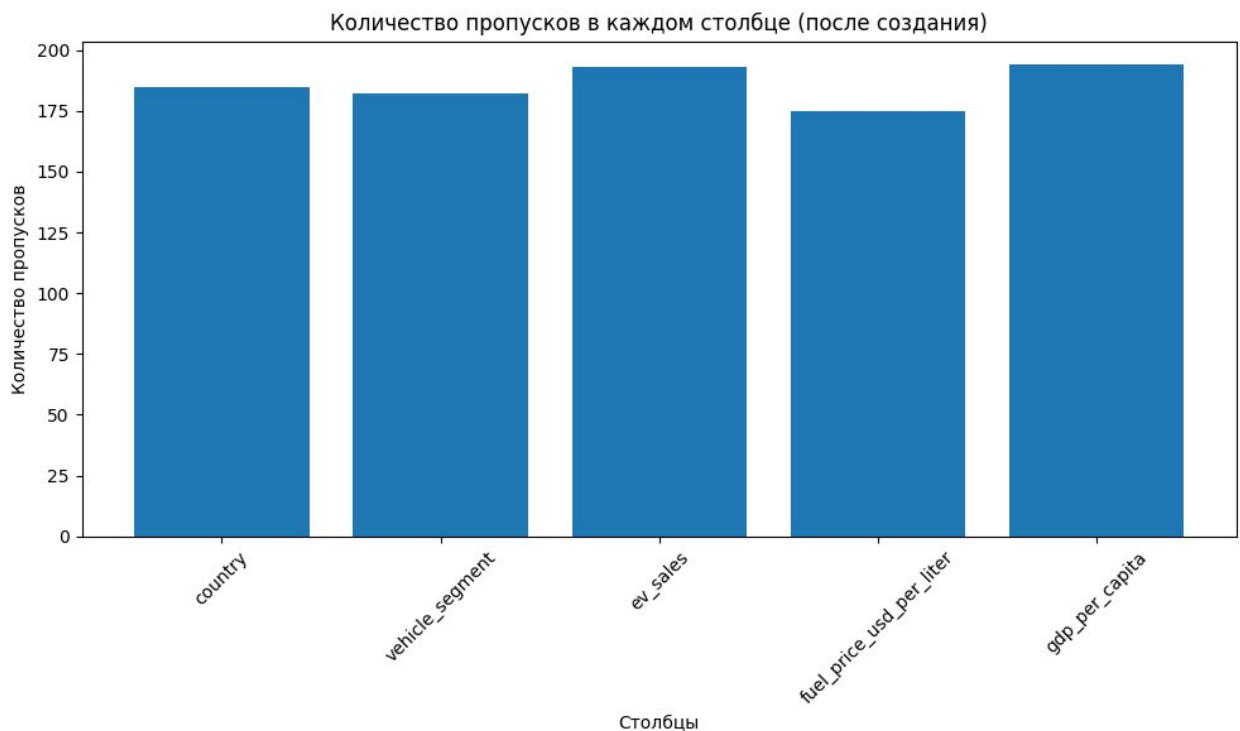


Таблица пропущенных значений после создания:

	Количество пропусков	Процент пропусков
gdp_per_capita	194	16.166667
ev_sales	193	16.083333
country	185	15.416667
vehicle_segment	182	15.166667
fuel_price_usd_per_liter	175	14.583333

После искусственного создания пропуска можем увидеть примерное 15% соотношение пропуска в каждой колонке, значения которых приблизительно составляют 190 ед. пропусков.

Применение различных методов заполнения пропусков

Существуют различные методы заполнения пропуска: среднее, медианное и мода значеине. Заполним пропуски тремя разными способами и получим следующие количества пропусков:

Сравнение методов заполнения пропусков:

Оригинальный датасет с пропусками - всего пропусков: 929

После заполнения средним (только числовые) - всего пропусков (оставшиеся категориальные): 367

После заполнения медианой (только числовые) - всего пропусков (оставшиеся категориальные): 367

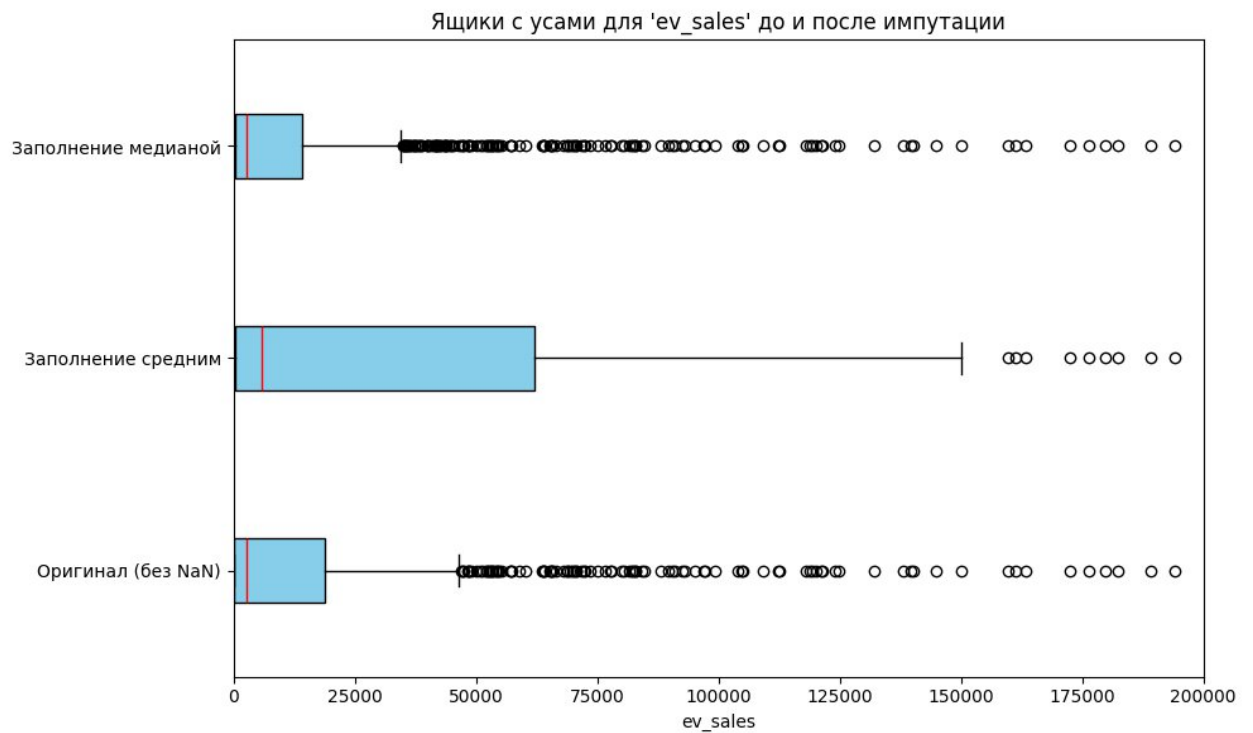
После заполнения модой (только категориальные) - всего пропусков (оставшиеся числовые): 562

Стоит обратить внимание, каждый из этих методов применялся только к одному типу признаков (числовым или категориальным).

Поэтому в 'df_mean_imputed' и 'df_median_imputed' остались пропуски в категориальных признаках, а в 'df_mode_imputed' остались пропуски в числовых признаках.

Сравнение распределений после импутации

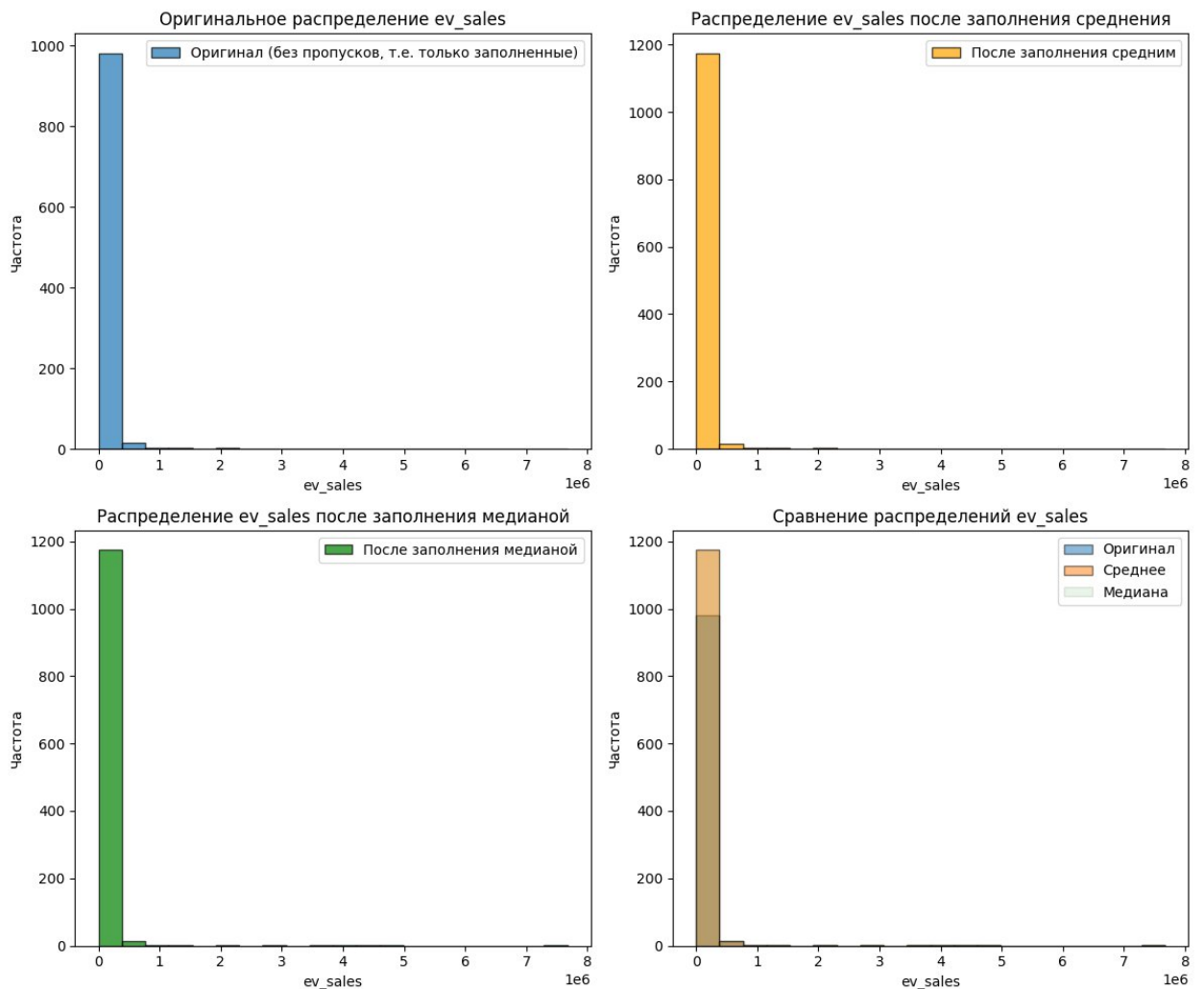
Выберем один числовой признак для демонстрации (например, ev_sales, так как в нем были созданы пропуски)



По графику можно заметить, что признак `ev_sales` демонстрирует сильно смещенное распределение с выраженными выбросами (среднее значительно выше медианы). Однако заполнение средним значением искажает распределение: медиана сдвигается вверх, а стандартное отклонение уменьшается, создавая искусственный пик в данных.

Подытожим, заполнение медианой лучше сохраняет центральную тенденцию исходного распределения (медиана остается неизменной) и менее чувствительно к выбросам.

Сравним распределение одного из признаков до и после импутации (гистограммы)



Как мы обсуждали, гистограммы наглядно демонстрируют:

- Оригинальное распределение 'ev_sales': Показывает сильное смещение вправо, с высокой концентрацией данных на низких значениях и длинным 'хвостом' в сторону высоких продаж EV.
- Заполнение средним значением: Создает четкий, искусственный пик в распределении в районе среднего значения, что значительно искажает исходную форму и плотность данных.
- Заполнение медианой: Гораздо лучше сохраняет исходную форму распределения, лишь немного увеличивая плотность в области медианы, не создавая при этом ложных концентраций.

Таким образом, визуальный анализ подтверждает, что для 'ev_sales' медианная импутация является предпочтительной, поскольку она меньше искажает естественное распределение данных.

Обоснование выбора метода для каждого типа признаков

Выводы по устранению пропусков:

- Среднее значение: Хорошо работает для нормально распределенных данных, но чувствительно к выбросам.
- Медиана: Более устойчива к выбросам, лучше подходит для асимметрично распределенных данных.
- Мода: Используется для категориальных переменных.

Для этого датасета, поскольку в нем нет пропусков, все методы дают одинаковый результат. В реальных условиях для числовых признаков в экономическом и социальном контексте лучше использовать медиану, а для категориальных - моду.

Кодирование категориальных признаков

Анализ категориальных переменных

Проверим уникальные значения категориальных признаков и получим следующее:

```
1 # Проверим уникальные значения категориальных признаков
2 print("Уникальные значения категориальных признаков:")
3 for col in categorical_cols:
4     print(f"{col}: {df_final[col].unique()}")
5     print(f"Количество уникальных значений: {df_final[col].nunique()}")
6     print()
```

Уникальные значения категориальных признаков:

country: ['Switzerland' 'Australia' 'Austria' 'Belgium' 'Brazil' 'Canada' 'China' 'France' 'Germany' 'India' 'Indonesia' 'Italy' 'Japan' 'Mexico' 'Netherlands' 'Norway' 'Poland' 'Portugal' 'South Korea' 'Spain' 'Sweden' 'Thailand' 'Turkey' 'United Kingdom' 'United States']

Количество уникальных значений: 25

region: ['Oceania' 'Europe' 'South America' 'North America' 'APAC']

Количество уникальных значений: 5

vehicle_segment: ['commercial' 'premium' 'mass_market']

Количество уникальных значений: 3

powertrain_type: ['ICE' 'BEV']

Количество уникальных значений: 2

Применение различных методов кодирования

```
1 # Подготовим копию датасета для кодирования
2 df_encoded = df_final.copy()
3
4 # 1. Label Encoding (для упорядоченных категорий)
5 df_label_encoded = df_final.copy()
6 label_encoders = {}
7
8 for col in categorical_cols:
9     le = LabelEncoder()
10    df_label_encoded[col] = le.fit_transform(df_label_encoded[col].astype(str))
11    label_encoders[col] = le
12
13 print("Label Encoding результат:")
14 for col in categorical_cols:
15     print(
16         f"{col}: {dict(zip(label_encoders[col].classes_, label_encoders[col].transform(label_encoders[col].classes_))}"
17     )
18
19 # 2. One-Hot Encoding (для неупорядоченных категорий)
20 df_onehot_encoded = pd.get_dummies(
21     df_final, columns=categorical_cols, prefix=categorical_cols
22 )
23
24 print(
25     f"\nOne-Hot Encoding: количество признаков до {df_final.shape[1]} -> после {df_onehot_encoded.shape[1]}"
26 )
27 print(
28     f"Пример новых столбцов после One-Hot Encoding: {df_onehot_encoded.columns[df_final.shape[1]:df_onehot_encoded.shape[1]:df_onehot_encoded.shape[1]]}"
29 )
```

Label Encoding результат:

country: {'Australia': np.int64(0), 'Austria': np.int64(1), 'Belgium': np.int64(2), 'Brazil': np.int64(3), 'Canada': np.int64(4), 'China': np.int64(5), 'France': np.int64(6), 'Germany': np.int64(7), 'India': np.int64(8), 'Indonesia': np.int64(9), 'Italy': np.int64(10), 'Japan': np.int64(11), 'Mexico': np.int64(12), 'Netherlands': np.int64(13), 'Norway': np.int64(14), 'Poland': np.int64(15), 'Portugal': np.int64(16), 'South Korea': np.int64(17), 'Spain': np.int64(18), 'Sweden': np.int64(19), 'Switzerland': np.int64(20), 'Thailand': np.int64(21), 'Turkey': np.int64(22), 'United Kingdom': np.int64(23), 'United States': np.int64(24)}

region: {'APAC': np.int64(0), 'Europe': np.int64(1), 'North America': np.int64(2), 'Oceania': np.int64(3), 'South America': np.int64(4)}

vehicle_segment: {'commercial': np.int64(0), 'mass_market': np.int64(1), 'premium': np.int64(2)}

powertrain_type: {'BEV': np.int64(0), 'ICE': np.int64(1)}

One-Hot Encoding: количество признаков до 22 -> после 53

Пример новых столбцов после One-Hot Encoding: ['country_Canada', 'country_China', 'country_France', 'country_Germany', 'country_India', 'country_Indonesia', 'country_Italy', 'country_Japan', 'country_Mexico', 'country_Netherlands', 'country_Norway', 'country_Poland', 'country_Portugal', 'country_South Korea', 'country_Spain', 'country_Sweden', 'country_Switzerland', 'country_Thailand', 'country_Turkey', 'country_United Kingdom', 'country_United States', 'region_Europe', 'region_North America', 'region_Oceania', 'region_South America', 'vehicle_segment_mass_market', 'vehicle_segment_premium', 'powertrain_type_BEV', 'powertrain_type_ICE']

Label Encoding результат:

country: {'Australia': np.int64(0), 'Austria': np.int64(1), 'Belgium': np.int64(2), 'Brazil': np.int64(3), 'Canada': np.int64(4), 'China': np.int64(5), 'France': np.int64(6), 'Germany': np.int64(7), 'India': np.int64(8), 'Indonesia': np.int64(9), 'Italy': np.int64(10), 'Japan': np.int64(11), 'Mexico': np.int64(12), 'Netherlands': np.int64(13), 'Norway': np.int64(14), 'Poland': np.int64(15), 'Portugal': np.int64(16), 'South Korea': np.int64(17), 'Spain': np.int64(18), 'Sweden': np.int64(19), 'Switzerland': np.int64(20), 'Thailand': np.int64(21), 'Turkey': np.int64(22), 'United Kingdom': np.int64(23), 'United States': np.int64(24)}

region: {'APAC': np.int64(0), 'Europe': np.int64(1), 'North America': np.int64(2), 'Oceania': np.int64(3), 'South America': np.int64(4)}

vehicle_segment: {'commercial': np.int64(0), 'mass_market': np.int64(1), 'premium': np.int64(2)}

```
'premium': np.int64(2)}
```

```
powertrain_type: {'BEV': np.int64(0), 'ICE': np.int64(1)}
```

One-Hot Encoding: количество признаков до 22 -> после 53

Пример новых столбцов после One-Hot Encoding: ['country_Canada', 'country_China', 'country_France', 'country_Germany', 'country_India', 'country_Indonesia', 'country_Italy', 'country_Japan', 'country_Mexico', 'country_Netherlands']...

Сравнение методов кодирования

Сравним размерности после разных методов кодирования таких как Label Encoding и One-Hot Encoding

```
Размер исходного датасета: (1200, 22)
Размер после Label Encoding: (1200, 22)
Размер после One-Hot Encoding: (1200, 53)
```

Примеры после Label Encoding:

	country	region	vehicle_segment	powertrain_type
0	20	3	0	1
1	0	3	0	1
2	0	3	2	1
3	0	3	0	1
4	0	3	1	1

Примеры столбцов после One-Hot Encoding:

	country_Australia	country_Austria	country_Belgium	country_Brazil	\
0	False	False	False	False	
1	True	False	False	False	
2	True	False	False	False	
3	True	False	False	False	
4	True	False	False	False	

	country_Canada	country_China
0	False	False
1	False	False
2	False	False
3	False	False
4	False	False

Обоснование выбора метода кодирования

Выводы по кодированию категориальных признаков:

– Label Encoding:

Преимущества: Сохраняет порядок (если он есть), не увеличивает

размерность данных

Недостатки: Может ввести модель в заблуждение, если категории не упорядочены

Использование: Для упорядоченных категорий или когда важна компактность

– One-Hot Encoding:

Преимущества: Не вводит искусственный порядок, хорошо работает с большинством алгоритмов

Недостатки: Увеличивает размерность данных, особенно при большом количестве уникальных значений

Использование: Для неупорядоченных категорий

Для нашего датасета будем использовать One-Hot Encoding, так как наши категориальные переменные (страна, регион, сегмент транспорта) не имеют естественного порядка.

Нормализация числовых признаков

Применение различных методов масштабирования

Существуют несколько вида методов нормализации: Min-Max Scaling и Standard. Рассмотрим их.

```
1 # Выберем числовые признаки для нормализации
2 numerical_features = df_final_encoded.select_dtypes(
3     include=[np.number]
4 ).columns.tolist()
5
6 # Проверим статистику до нормализации
7 print("Статистика числовых признаков до нормализации:")
8 df_final_encoded[numerical_features].describe().round(3)
```

Статистика числовых признаков до нормализации:

	year	ev_sales	petrol_car_sales	diesel_car_sales	total_vehicle_sales	ev_market_share	chi
count	1200.000	1200.000	1.200000e+03	1200.000	1.200000e+03	1200.000	
mean	2017.500	52455.780	7.733063e+05	140936.883	9.778455e+05	6.328	
std	4.612	363405.001	1.756934e+06	291347.174	2.211799e+06	13.232	
min	2010.000	5.000	1.727000e+03	150.000	1.748000e+04	0.000	
25%	2013.750	392.000	8.870500e+04	19510.750	1.300465e+05	0.078	
50%	2017.500	2636.000	2.294785e+05	51361.000	2.905130e+05	0.830	
75%	2021.250	14129.750	6.385320e+05	111191.000	7.624482e+05	5.832	
max	2025.000	7670056.000	1.477369e+07	2014595.000	1.884998e+07	95.000	


```

1 # 1. Min-Max Scaling (нормализация в диапазон [0, 1])
2 scaler_minmax = MinMaxScaler()
3 df_minmax_scaled = df_final_encoded.copy()
4 df_minmax_scaled[numerical_features] = scaler_minmax.fit_transform(
5     df_final_encoded[numerical_features]
6 )
7
8 # 2. Standardization (Z-score нормализация)
9 scaler_standard = StandardScaler()
10 df_standard_scaled = df_final_encoded.copy()
11 df_standard_scaled[numerical_features] = scaler_standard.fit_transform(
12     df_final_encoded[numerical_features]
13 )
14
15 print(f"\nКоличество числовых признаков для нормализации: {len(numerical_features)}")

```

Количество числовых признаков для нормализации: 18

Сравнение методов нормализации

```

1 # Сравним статистику после разных методов нормализации
2 print("Статистика после Min-Max нормализации:")
3 df_minmax_scaled[numerical_features].describe().round(3)

```

Статистика после Min-Max нормализации:

	year	ev_sales	petrol_car_sales	diesel_car_sales	total_vehicle_sales	ev_market_share	char
count	1200.000	1200.000	1200.000	1200.000	1200.000	1200.000	
mean	0.500	0.007	0.052	0.070	0.051	0.067	
std	0.307	0.047	0.119	0.145	0.117	0.139	
min	0.000	0.000	0.000	0.000	0.000	0.000	
25%	0.250	0.000	0.006	0.010	0.006	0.001	
50%	0.500	0.000	0.015	0.025	0.014	0.009	
75%	0.750	0.002	0.043	0.055	0.040	0.061	
max	1.000	1.000	1.000	1.000	1.000	1.000	

```

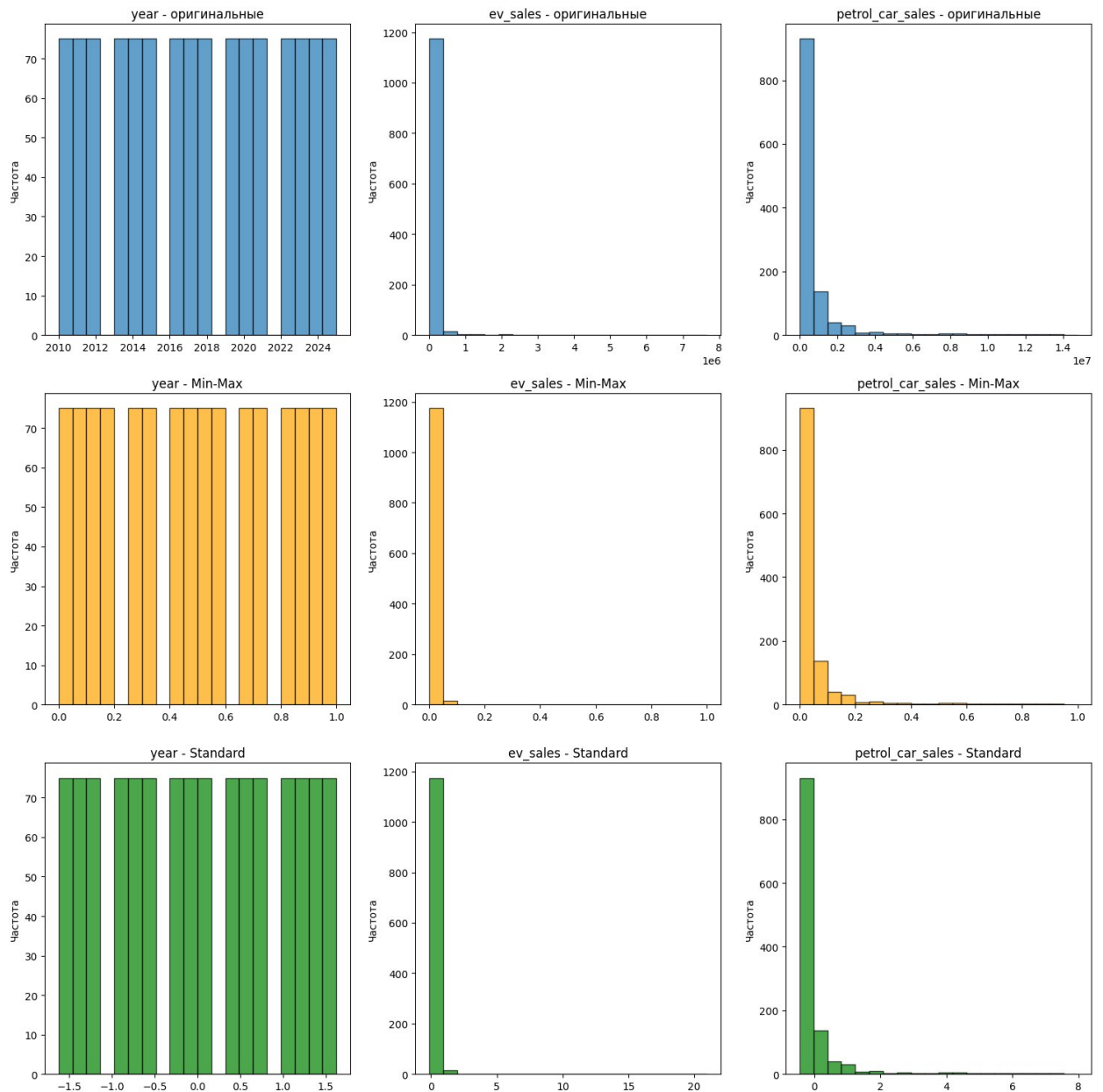
1 print("\nСтатистика после Standard нормализации:")
2 df_standard_scaled[numerical_features].describe().round(3)

```

Статистика после Standard нормализации:

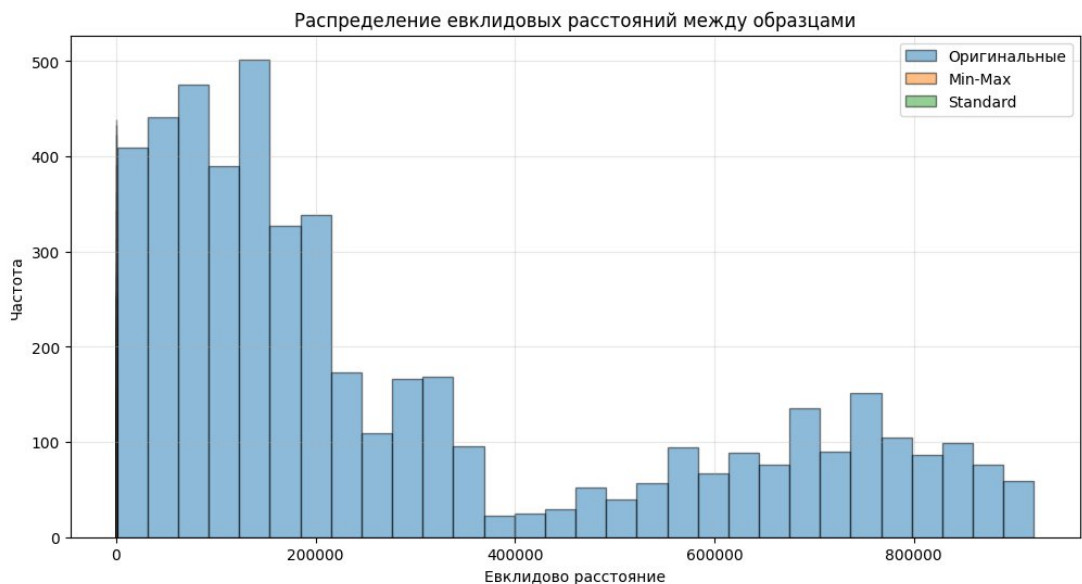
	year	ev_sales	petrol_car_sales	diesel_car_sales	total_vehicle_sales	ev_market_share	char
count	1200.000	1200.000	1200.000	1200.000	1200.000	1200.000	
mean	0.000	0.000	0.000	0.000	0.000	0.000	
std	1.000	1.000	1.000	1.000	1.000	1.000	
min	-1.627	-0.144	-0.439	-0.483	-0.434	-0.478	
25%	-0.813	-0.143	-0.390	-0.417	-0.383	-0.473	
50%	0.000	-0.137	-0.310	-0.308	-0.311	-0.416	
75%	0.813	-0.106	-0.077	-0.102	-0.097	-0.037	
max	1.627	20.970	7.972	6.434	8.084	6.704	

Рассмотрим визуализацию влияния нормализации (гистограммы)



Можно заметить, что после нормализации данные не искажаются совсем. Для убедительной информации построим гистограмму двух разных методов нормализации на расстояния и наложим поверх друг друга.

Рассчитаем евклидовы расстояния между первыми 100 образцами для разных методов, затем возьмем подмножество данных для демонстрации и рассчитаем расстояния.



По графику видно, что два метода нормализации не отличаются друг друга.

Обоснование выбора оптимального метода

Выводы по нормализации числовых признаков:

– Min-Max Scaling:

Преимущества: Сохраняет все свойства распределения, приводит все признаки к одинаковому диапазону $[0,1]$

Недостатки: Чувствителен к выбросам, так как использует минимум и максимум

Использование: Когда известны границы значений признаков, для нейронных сетей

– Standardization (Z-score):

Преимущества: Не чувствителен к выбросам, сохраняет форму распределения

Недостатки: Не приводит признаки к одинаковому диапазону

Использование: Когда неизвестны границы значений, для большинства алгоритмов машинного обучения

Для этого датасета, содержащего экономические и социальные показатели, более подходящим является StandardScaler, так как он более устойчив к выбросам, которые часто встречаются в таких данных.

Вывод

В ходе лабораторной работы был проведён полный цикл предобработки данных для анализа рынка электромобилей на основе датасета за 2010–2025 гг. (1200 наблюдений). В результате разведочного анализа (EDA) выявлено, что электромобили демонстрируют быстрый рост (~64% в год), но глобально остаются меньшинством (средняя доля 6.33%), а переход концентрируется в странах с развитой инфраструктурой, высокими доходами и жёсткой экологической политикой. Были изучены методы импутации пропущенных значений и определена оптимальная стратегия: медиана для числовых признаков (устойчива к выбросам в асимметричных распределениях, таких как `ev_sales`) и мода для категориальных. Для кодирования категориальных признаков выбран One-Hot Encoding, так как переменные (страна, регион, сегмент, тип двигателя) не имеют естественного порядка. При нормализации числовых признаков рекомендован `StandardScaler` как более устойчивый к выбросам метод по сравнению с `Min-Max Scaling`. Все проведённые этапы предобработки позволили подготовить датасет к дальнейшему построению моделей машинного обучения для прогнозирования перехода на электромобили.